CIS 678 - Machine Learning

– Neural Networks –

Instructor: Yong Zhuang

yong.zhuang@gvsu.edu

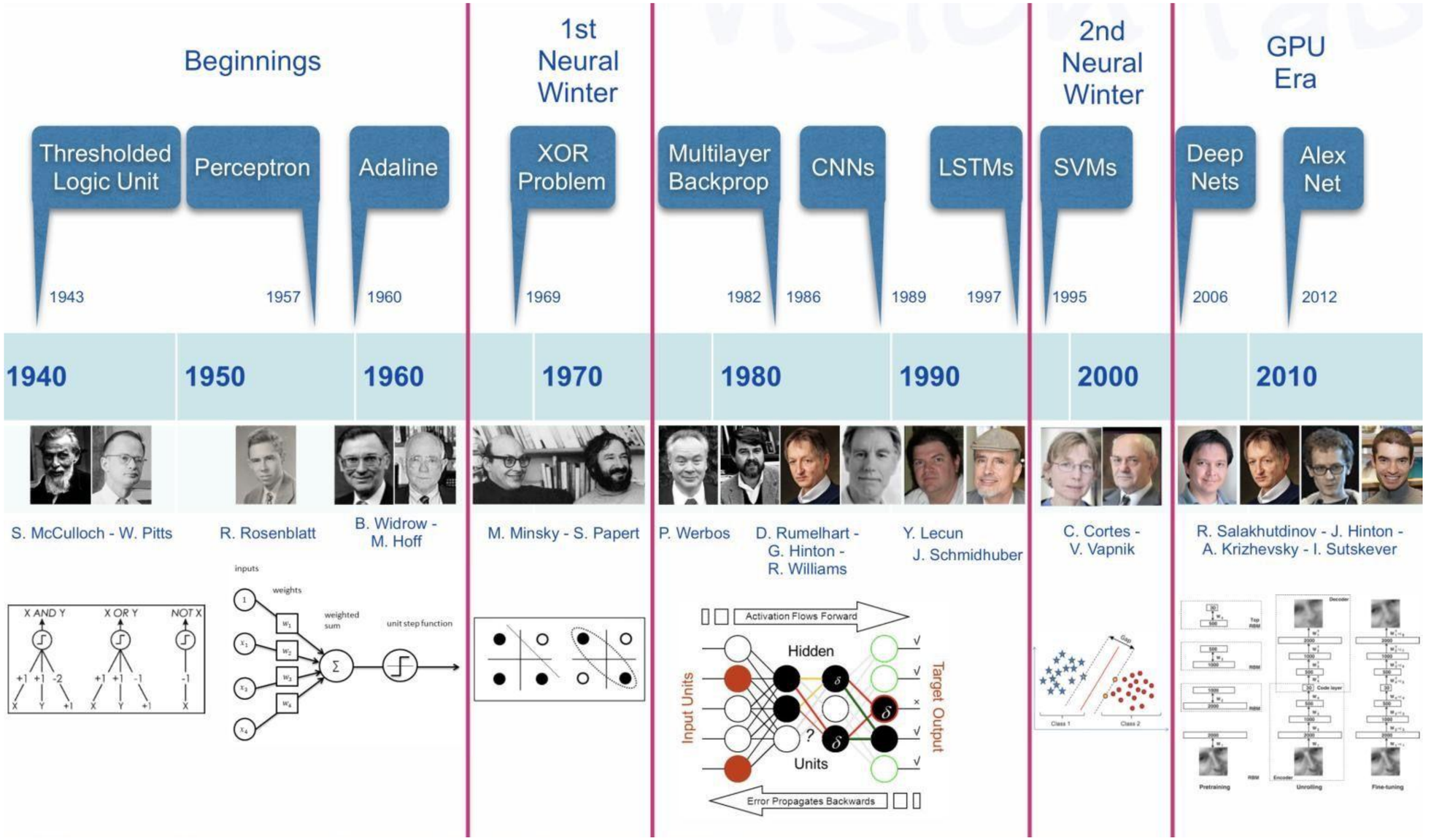
Last Class

- Generative vs. Discriminative Classifiers
- Why Logistic Regression?
- Sigmoid Function and Logistic Regression Model
- Linear Decision Boundary
- Maximum Likelihood Estimation
- Logistic Loss and Gradient Descent
- MAP and Regularization

Agenda

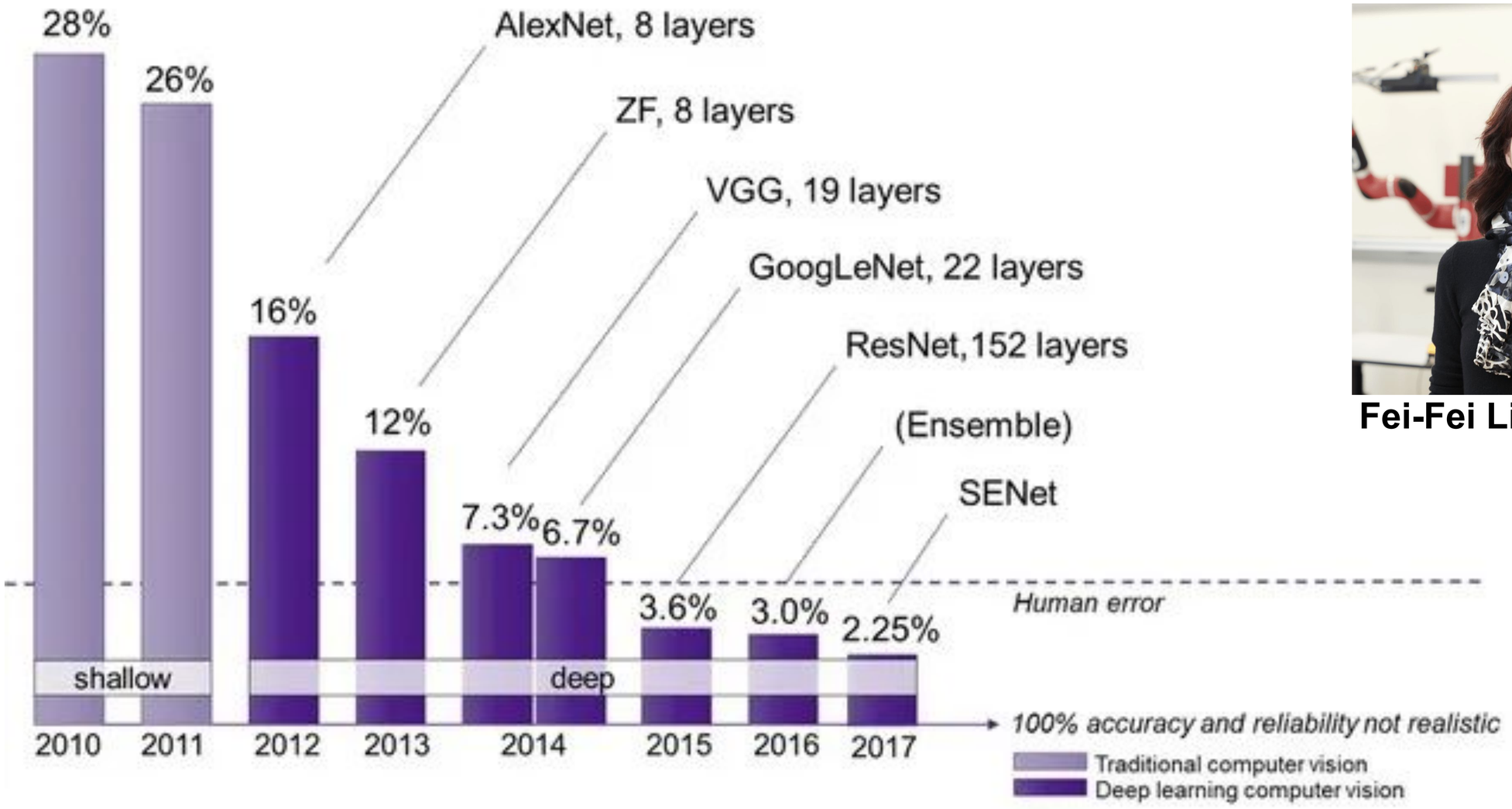
- Brief History and Motivation
- From Logistic Regression to a Neural Unit
- Neural Network Architecture
- Hidden Layers and Representations
- Activation Functions
- Why Nonlinearity Matters
- Forward Propagation
- Preview: How Do Neural Networks Learn?

A Brief History of Neural Networks



Source: https://i.pinimg.com/originals/6a/f0/3c/6af03c5026bb680ebe6d8db4bdbb8428.jpg

ImageNet Challenge





Fei-Fei Li — ImageNet

Source: https://semiengineering.com/new-vision-technologies-for-real-world-applications/

Beyond Image Recognition: AlphaGo and GPT

Master of Go Board Game Is Walloped by Google Computer Program

Share full article

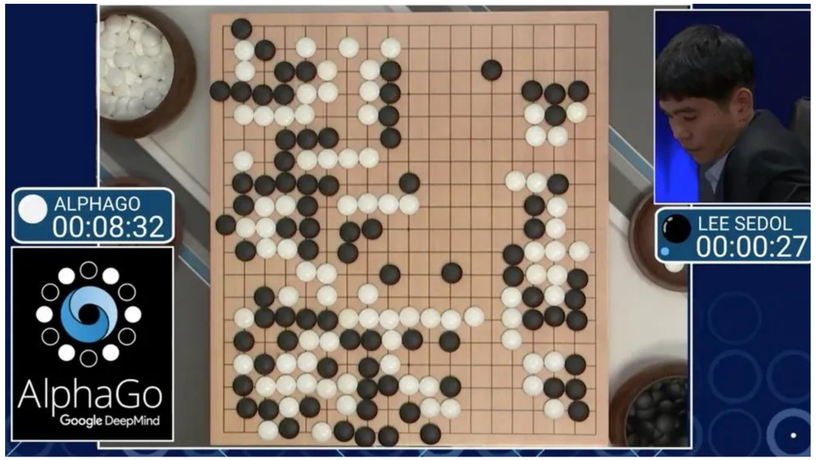
By Choe Sang-Hun and John Markoff

March 9, 2016

SEOUL, South Korea — Computer, one. Human, zero.

A Google computer program stunned one of the world's top players on Wednesday in a round of Go, which is believed to be the most complex board game ever created.

Source



Source

OpenAI announces ChatGPT successor GPT-4

14 March 2023

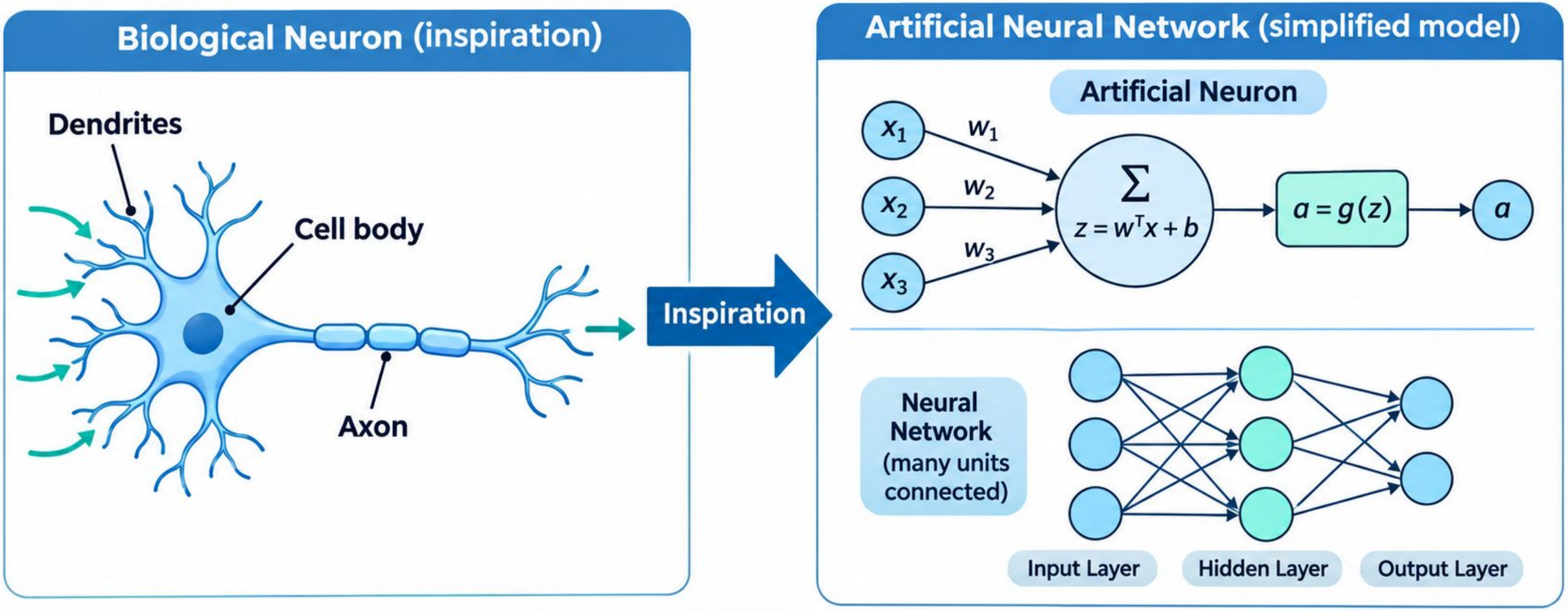
Share

By Ben Derico and Zoe Kleinman, BBC News



OpenAI has released GPT-4, the latest version of its hugely popular artificial intelligence chatbot ChatGPT.

From Biological Inspiration to Artificial Neural Networks



Biological neurons inspired artificial neural networks, but the artificial version is a simplified mathematical model.

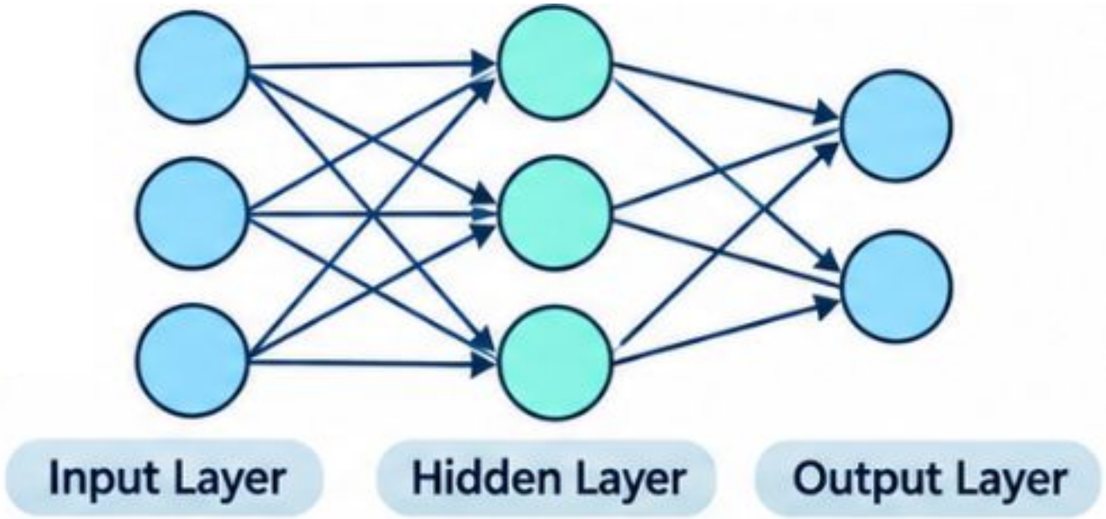
Key Components of a Neural Network

- Architecture
- Activation function
- Loss function
- Optimization
- Regularization

Architecture

A neural network architecture specifies its structure:

- Number of input units
- Number of hidden layers
- Number of units in each hidden layer
- Type of units / activation functions
- Connections between layers
- Number of output units



The architecture determines the form of the function

$$f(x;\theta)$$

where θ contains the parameters to be learned.

Activation function

For each neuron:

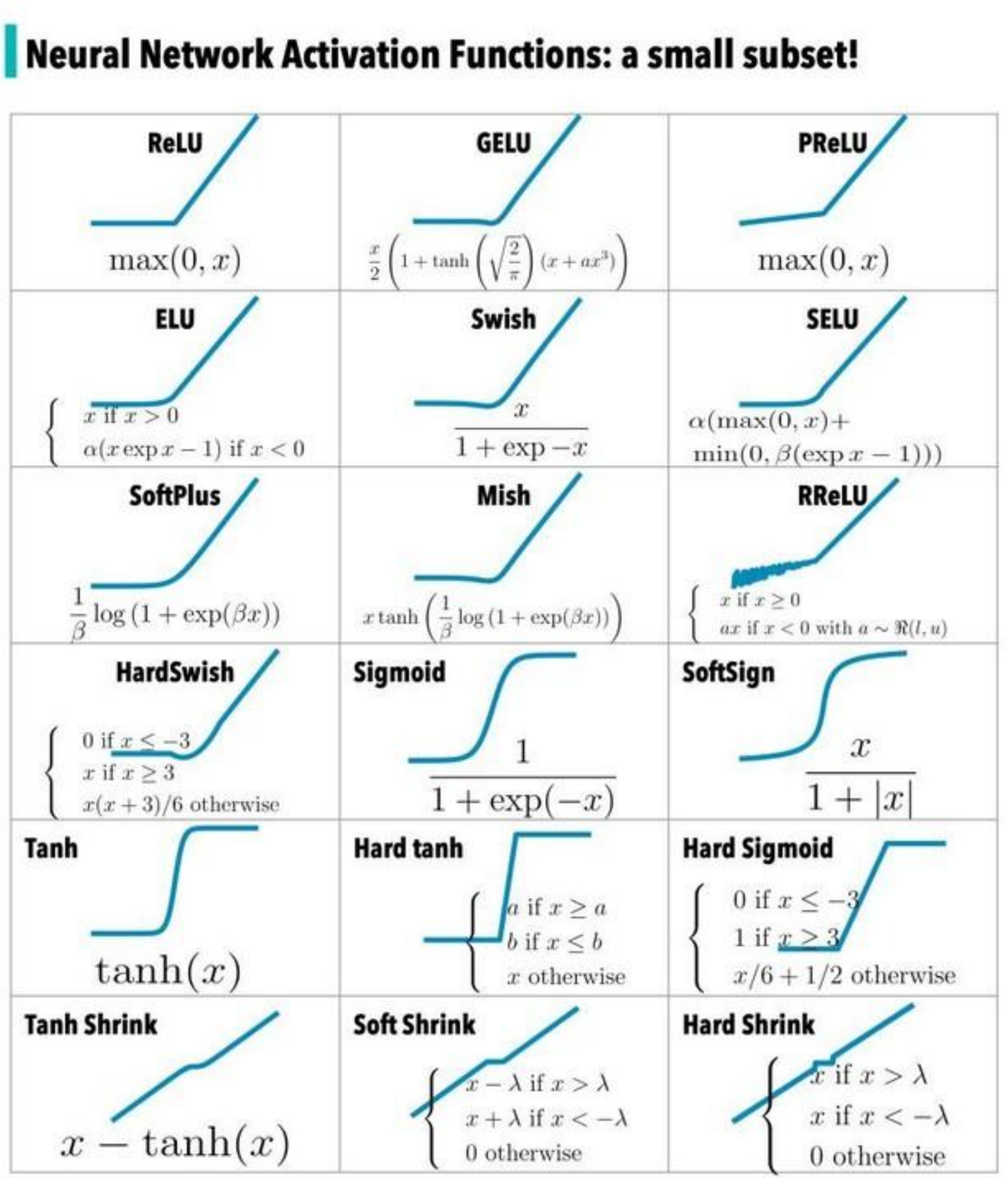
$$z = w^T x + b$$

then apply an activation function:

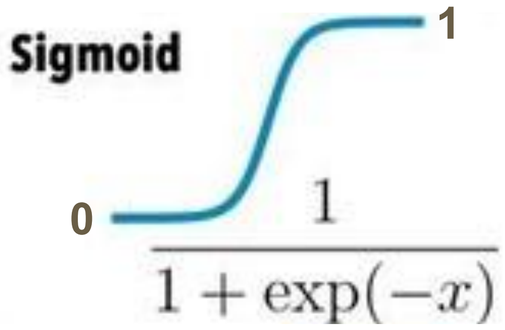
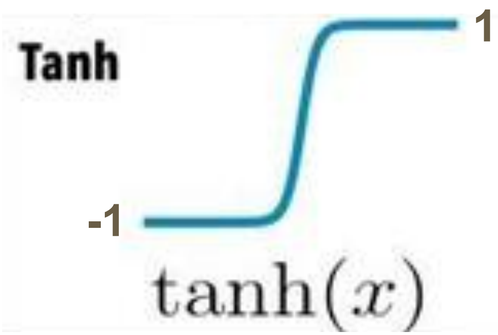
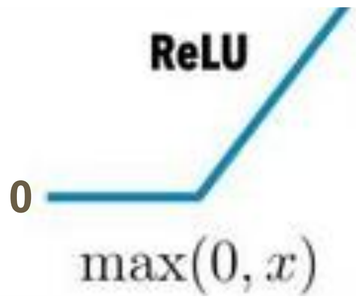
$$\alpha = f(z)$$

Why use an activation function?

- Controls the form of the neuron output
- Introduces **nonlinearity**, allows neural networks to learn more complex relationships
- Different layers may use different activation functions
- Good activation functions should be differentiable for optimization purpose



Activation function

Activation	Range	Main use	Cons
	(0,1)	useful for binary output	can suffer from vanishing gradients.
	(-1,1)	zero-centered;	can still suffer from vanishing gradients.
	[0,∞)	common for hidden layers; simple and efficient;	can have “dead” neurons when the unit stays in the zero-gradient region.

Why Do We Need Activation Functions?

Key idea: Without nonlinear activation functions, a multi-layer neural network is still a linear model.

- Suppose each layer is linear (no activation):

$$z^{(1)} = W^{(1)}\boldsymbol{x} + b^{(1)}$$
$$z^{(2)} = W^{(2)}z^{(1)} + b^{(2)}$$

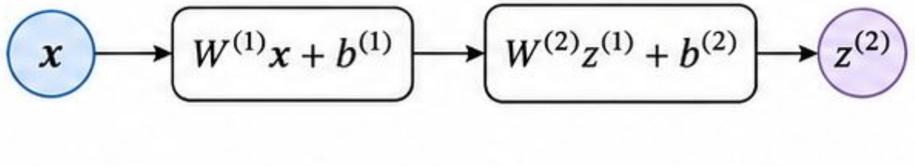
- Substitute $z^{(1)}$ into $z^{(2)}$:

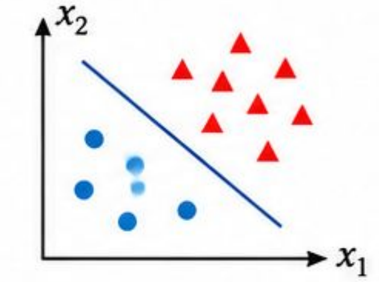
$$z^{(2)} = W^{(2)}(W^{(1)}\boldsymbol{x} + b^{(1)}) + b^{(2)}$$
$$= (W^{(2)}W^{(1)})\boldsymbol{x} + (W^{(2)}b^{(1)} + b^{(2)})$$

- This is still a linear function of $\boldsymbol{x}$.

Conclusion: Without a nonlinear activation function, stacking layers does not increase the model's expressive power.

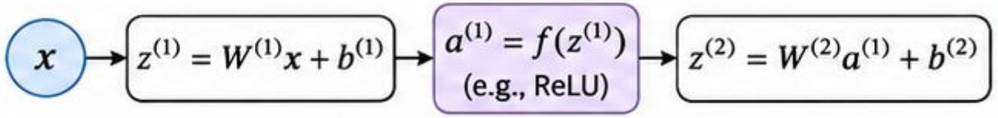
Linear network (no activation): still linear

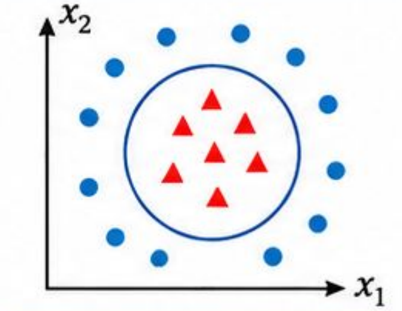




Linear decision boundary

Nonlinear network (with activation, e.g., ReLU): can model complex patterns





Nonlinear decision boundary

Loss Function: Measuring Prediction Error

Key idea: A loss function measures how different the model's prediction is from the true target.

- Denote the model prediction by $\hat{y}$ and the true target by y.
- The loss $L(\hat{y}, y)$ quantifies the prediction error for one example.
- On a dataset with n examples, we usually minimize the average loss:

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^{n} L(\hat{y}_i, y_i)$$

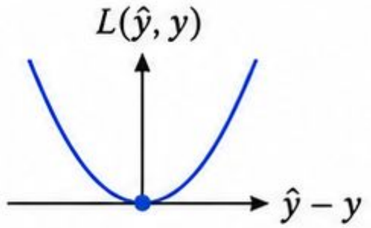
where θ denotes the model parameters (e.g., weights and biases).

Common Loss Functions

Regression
(e.g., real-valued output)

Mean Squared Error (MSE)

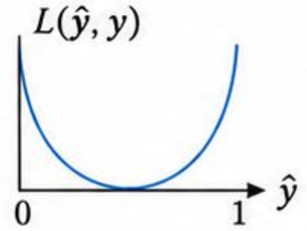
$$\mathcal{L}(\hat{y}, y) = (\hat{y} - y)^2$$



Classification
(e.g., binary or multi-class)

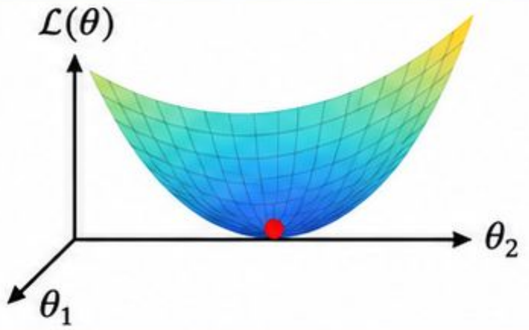
Cross-Entropy Loss
(Binary Classification)

$$\mathcal{L}(\hat{y}, y) = -\left[y \log \hat{y} + (1 - y) \log(1 - \hat{y})\right]$$



Intuition

- The loss is always non-negative.
- A smaller loss means a better fit to the data.
- Training a neural network means finding parameters θ that minimize the loss $\mathcal{L}(\theta)$.



Optimization (Learning)

Key idea: Use an optimization algorithm to adjust the model parameters to minimize the loss function.

- **Objective:** find parameters θ that minimize the loss $\mathcal{L}(\theta)$:

$$\theta^* = \arg\min_{\text{theta}} \mathcal{L}(\theta)$$

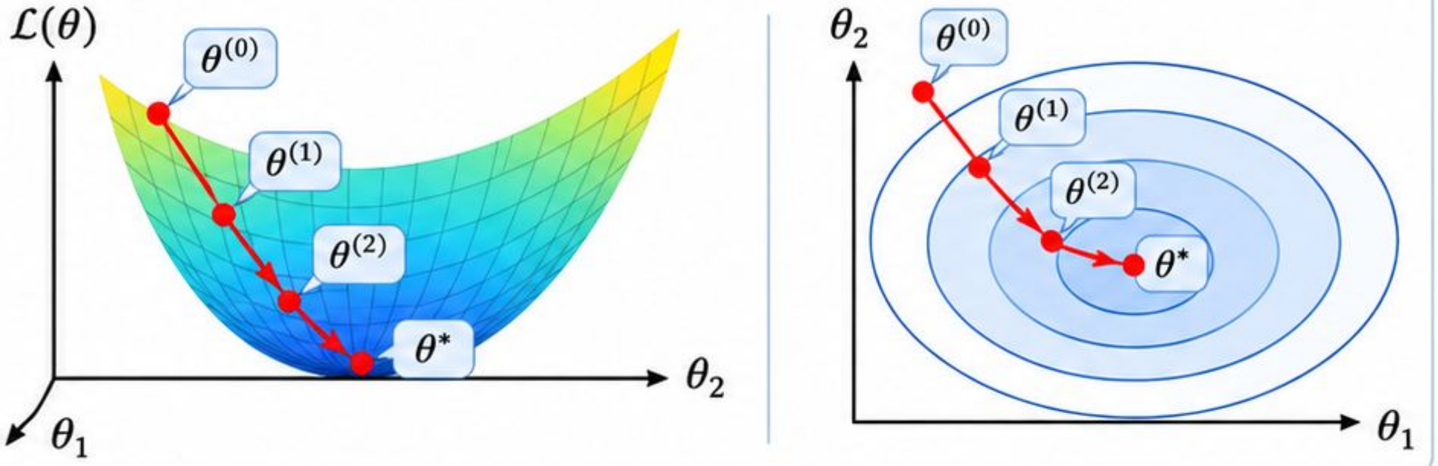
- **Method:** Gradient Descent (GD)
  - Compute the gradient $\nabla\mathcal{L}(\theta)$.
  - Update the parameters:

$$\theta \leftarrow \theta - \eta\nabla\mathcal{L}(\theta)$$

  - Repeat until convergence.
    (η is the learning rate.)

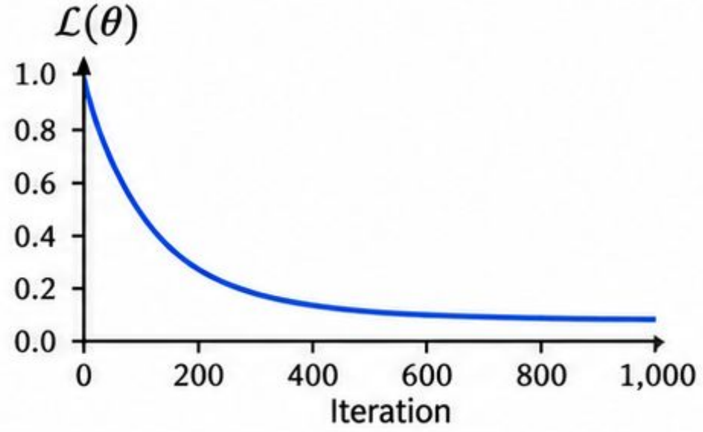
Note: In practice, we often use mini-batch gradient descent (or its variants) for efficiency.

Gradient Descent: Intuition



Algorithm: Gradient Descent (for Neural Networks)

1. Initialize parameters θ (e.g., small random values).
2. Compute the gradient $\nabla\mathcal{L}(\theta)$ (using backpropagation).
3. Update $\theta \leftarrow \theta - \eta\nabla\mathcal{L}(\theta)$.
4. Repeat until the loss converges.



Compute gradients $\nabla_\theta L(\theta)$ using backpropagation

Example: Logistic Regression

Objective (Logistic Loss)

$$\mathcal{L}(\mathbf{w}, b) = -\sum_{i=1}^{n} \left[y_i \log p_i + (1 - y_i) \log (1 - p_i) \right]$$

$$p_i = \sigma(\mathbf{w}^T \mathbf{x}_i + b)$$

Gradients

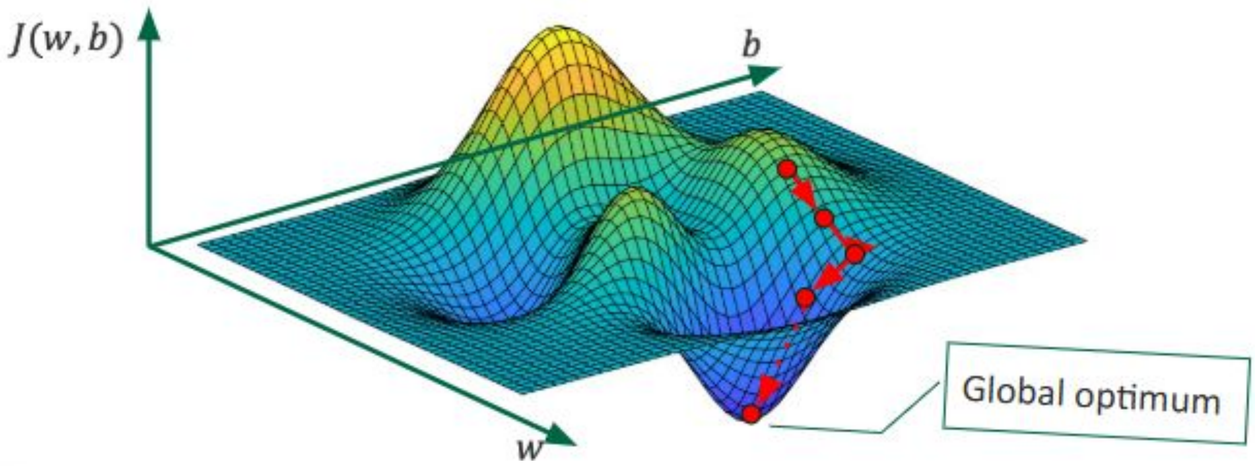
$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \sum_{i=1}^{n} (p_i - y_i) \mathbf{x}_i \qquad \frac{\partial \mathcal{L}}{\partial b} = \sum_{i=1}^{n} (p_i - y_i)$$

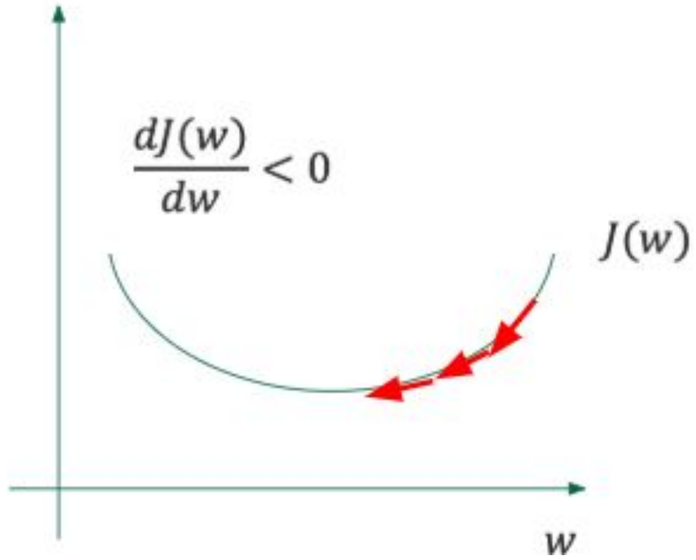
Gradient Descent Update

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \mathbf{w} - \eta \sum_{i=1}^{n} (p_i - y_i) \mathbf{x}_i$$

$$b \leftarrow b - \eta \frac{\partial \mathcal{L}}{\partial b} = b - \eta \sum_{i=1}^{n} (p_i - y_i)$$

η: learning rate (a small positive number)





Regularization

Key idea: Use regularization and training techniques to control model complexity, improve generalization, and reduce overfitting.

- **Regularized loss:**

$$\mathcal{L}_{\text{reg}}(\theta) = \mathcal{L}(\theta) + \lambda\, \Omega(\theta)$$

  - $\Omega(\theta)$: regularization term (penalty)
  - $\lambda \geq 0$: regularization strength (controls the penalty)

- **Common penalties:**
  - L2 (Ridge): $\Omega(\theta) = \|\theta\|_2^2 = \sum_j \theta_j^2$
  - L1 (Lasso): $\Omega(\theta) = \|\theta\|_1 = \sum_j |\theta_j|$

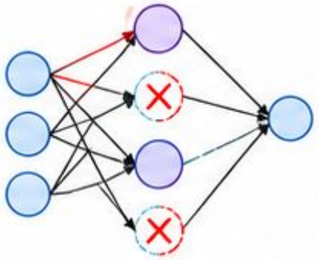
Note: These techniques can be combined (e.g., L2 + dropout + early stopping) for better generalization.

L1 vs. L2 Regularization (Weight Penalties)

Method	Regularization Term $\Omega(\theta)$	Effect
L2 (Ridge)	$\|\theta\|_2^2 = \sum_j \theta_j^2$	■ Penalizes large weights. ■ Tends to shrink weights toward zero. ■ Keeps most weights non-zero.
L1 (Lasso)	$\|\theta\|_1 = \sum_j \|\theta_j\|$	■ Penalizes the absolute values of weights. ■ Can drive some weights exactly to zero (acts as feature selection).

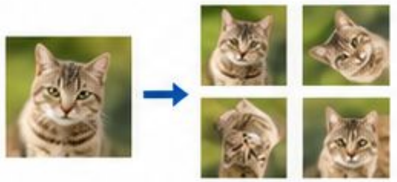
Other Common Regularization Techniques

Dropout



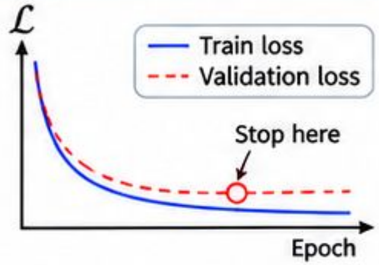
- Randomly "drop" neurons during training (e.g., $p = 0.5$).
- Reduces co-adaptation and overfitting.

Data Augmentation



- Create additional training examples (e.g., flip, rotate, crop, color jitter).
- Increases data diversity and generalization.

Early Stopping



- Monitor validation loss during training.
- Stop when it stops improving to avoid overfitting.

Batch Normalization (BN)

$$\hat{z} = \frac{z - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y = \gamma \hat{z} + \beta$$

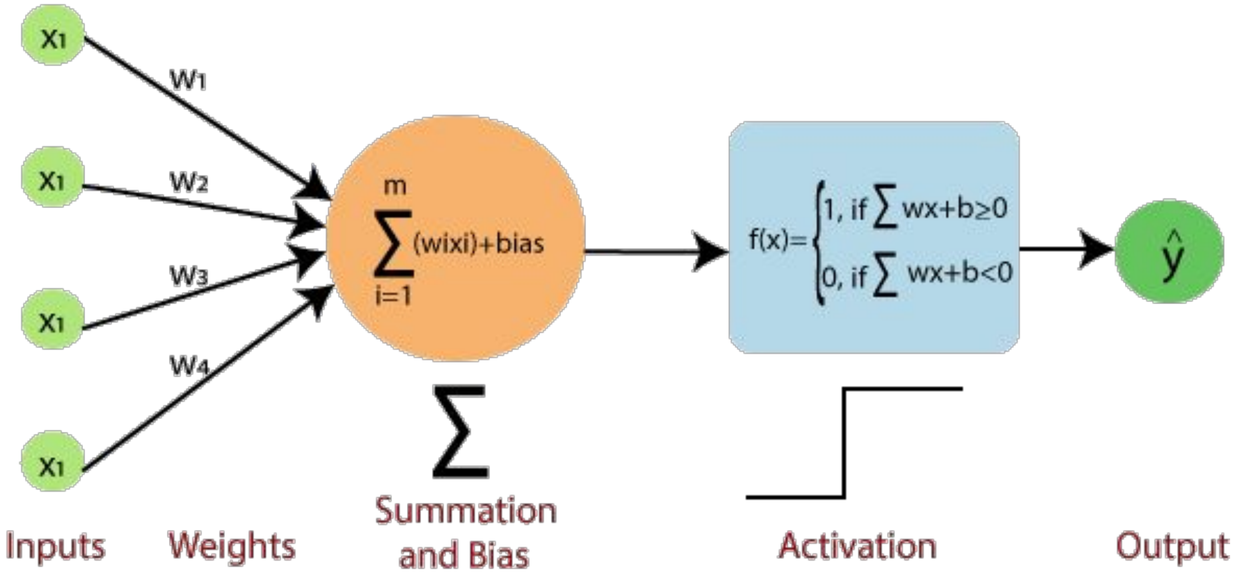
- Normalize layer outputs using mini-batch statistics (μ_B, σ_B^2).
- Stabilizes and accelerates training, and can improve generalization.

Optimizer Choice (e.g., SGD)

- **SGD** (Stochastic Gradient Descent): updates parameters using a mini-batch of data (computationally efficient).
- **Common variants:** Momentum, RMSprop, Adam (often faster and more stable than vanilla SGD).

Single Unit: Perceptron

- **Architecture:**
  - A single neuron: A Perceptron can be viewed as a single neural-network unit.
- **Activation function**
  - Training: identity function
  - Inference: $\hat{y} = \text{sign}(z)$
- **Loss function**
  - $l(y, \hat{y}) = \max(0, -y\hat{y})$
- **Optimization**
  - $w \leftarrow w + \eta y^{(i)} x^{(i)}$, for a misclassified training data point $(x^{(i)}, y^{(i)})$, i.e., $y^{(i)} w^T x^{(i)} \leq 0$
  - $b \leftarrow b + \eta y^{(i)}$
- η: learning rate

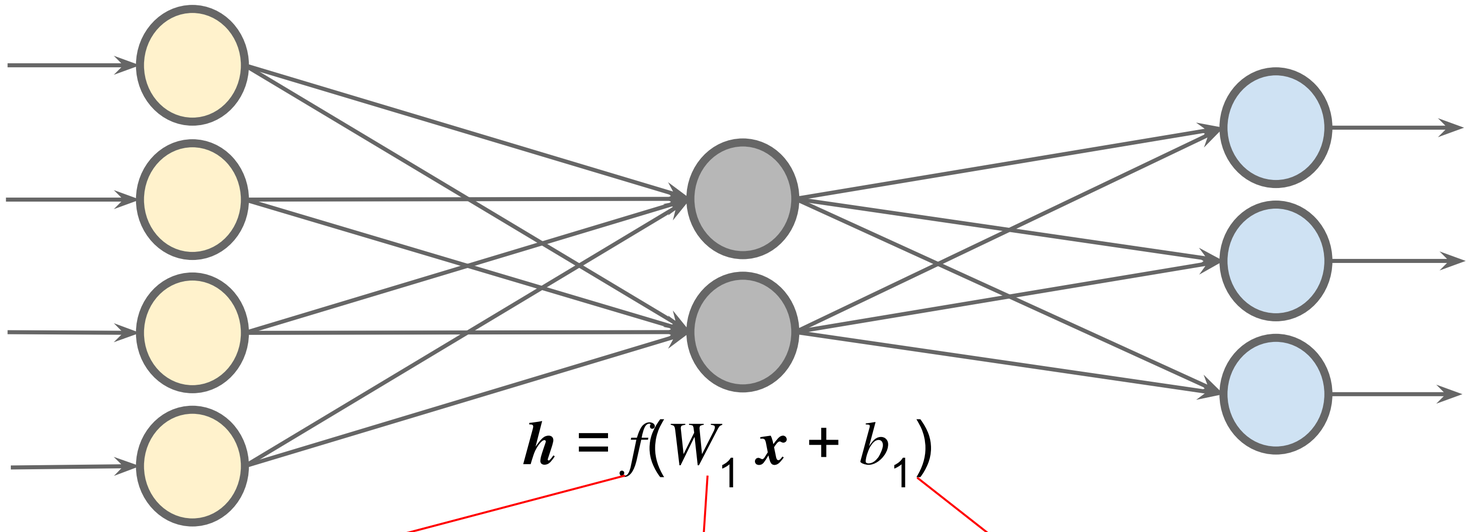


Single Unit: Logistic Regression

- **Architecture:**
  - A single neuron
- **Activation function**
  - Sigmoid function
- **Loss function**
  - $l(y, \hat{y}) = -y log\hat{y} - (1-y) log(1-log\hat{y})$
  - **Note** $\hat{y}$ is the predicted probability of taking class 1.
- **Optimization**
  - $\boldsymbol{w} \leftarrow \boldsymbol{w} + \eta(y^{(i)} - \sigma(w^T x^{(i)}))x^{(i)}$, for a training data point $(x^{(i)}, y^{(i)})$.
- η: learning rate

A Multi-Layer Feed-Forward Neural Network

- **A two-layer network**



$$W_1 \in \mathbb{R}^{4\times 2} \begin{Bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \\ w_{31} & w_{32} \\ w_{41} & w_{42} \end{Bmatrix}$$

$$W_2 \in \mathbb{R}^{2\times 3} \begin{Bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{Bmatrix}$$

How A Multi-Layer Neural Network Works

- The **inputs** to the network correspond to the attributes measured for each training tuple
- Inputs are fed simultaneously into the units making up the **input layer**
- They are then weighted and fed simultaneously to a **hidden layer**
  - The number of hidden layers is arbitrary
- The weighted outputs of the last hidden layer are input to units making up the **output layer**, which emits the network's prediction
- The network is **feed-forward**: None of the weights cycles back to an input unit or to an output unit of a previous layer
- From a math point of view, networks perform **nonlinear regression**: *Given enough hidden units and enough training samples, they can closely approximate any continuous function*

Learning by Backpropagation

- Iteratively process a set of training tuples & compare the network's prediction with the actual known target value
- For each training tuple, the weights are modified to **minimize the loss function** between the network's prediction and the actual target value, say **mean squared error**
  - Stochastic gradient descent + chain rule
- Modifications are made in the "**backwards**" direction: from the output layer, through each hidden layer down to the first hidden layer, hence "**backpropagation**"

Recap: Chain Rule

- The **chain rule** is a formula that expresses the derivative of the composition of two differentiable functions *f* and *g* in terms of the derivatives of *f* and *g*.

  If $y = f(u)$ and $u = g(x)$ are both differentiable functions, then

$$\frac{dy}{dx} = \frac{dy}{du}\frac{du}{dx}$$

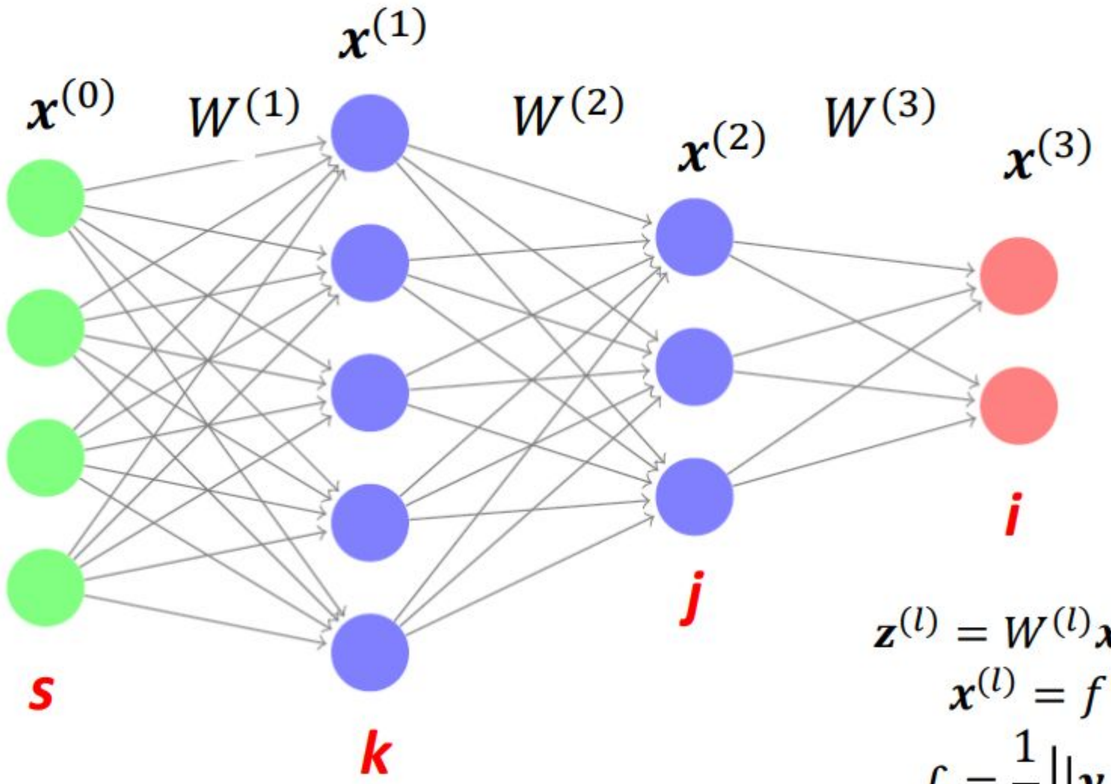
$\frac{dy}{dx}$ = derivative of y with respect to x

$\frac{dy}{du}$ = derivative of y with respect to u

$\frac{du}{dx}$ = derivative of u with respect to x

Example

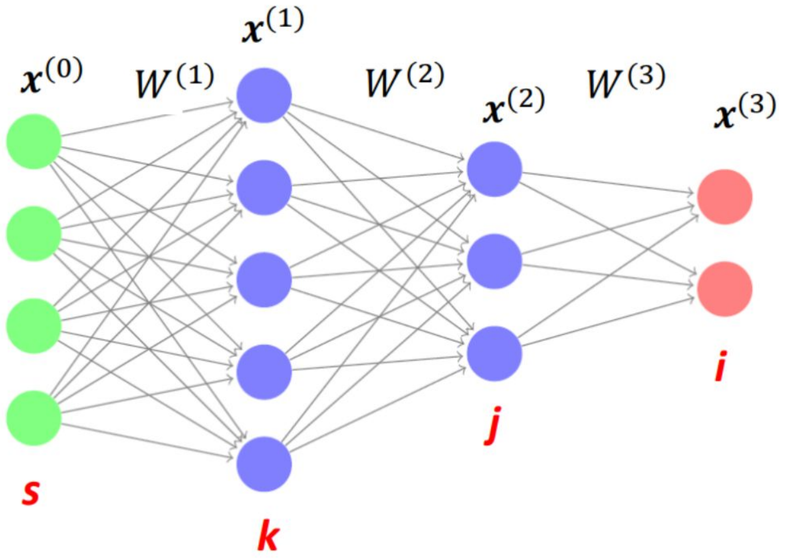
- Loss function: $\mathcal{L} = \frac{1}{2}||\boldsymbol{y} - \hat{\boldsymbol{y}}||^2$



$$\boldsymbol{z}^{(l)} = W^{(l)}\boldsymbol{x}^{(l-1)} + \boldsymbol{b}^{(l)}$$
$$\boldsymbol{x}^{(l)} = f^{(l)}\left(\boldsymbol{z}^{(l)}\right)$$
$$\mathcal{L} = \frac{1}{2}\left|\left|\boldsymbol{y} - \boldsymbol{x}^{(3)}\right|\right|^2$$

Gradient for Layer 3 (Last Layer)

- Stochastic gradient for $W_{ij}^{(3)}$ and $b_{ij}^{(3)}$



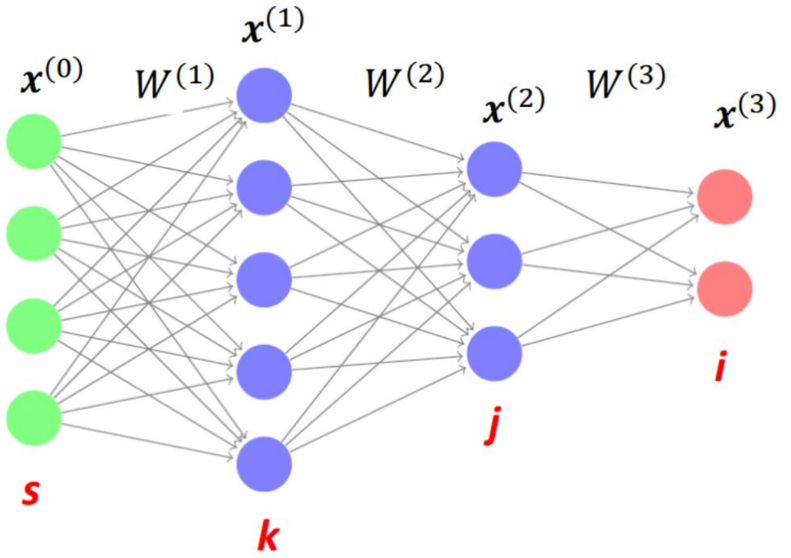
  - *Recall:*
    - $\mathcal{L} = \frac{1}{2}\left|\left|\boldsymbol{y} - \boldsymbol{x}^{(3)}\right|\right|^2 = \frac{1}{2}\sum_i \left(y_i - x_i^{(3)}\right)^2$
    - $x_i^{(3)} = f^{(3)}(z_i^{(3)})$
    - $z_i^{(3)} = \sum_j W_{ij}^{(3)} x_j^{(2)} + b_i^{(3)}$

- $$\frac{\partial \mathcal{L}}{\partial W_{ij}^{(3)}} = \frac{\partial \mathcal{L}}{\partial x_i^{(3)}} \frac{\partial x_i^{(3)}}{\partial z_i^{(3)}} \frac{\partial z_i^{(3)}}{\partial W_{ij}^{(3)}} = -\underbrace{(y_i - x_i^{(3)}) \mathrm{f}'^{(3)}\left(\mathrm{z}_\mathrm{i}^{(3)}\right)}_{\delta_i^{(3)}} \mathrm{x}_j^{(2)}$$

- $$\frac{\partial \mathcal{L}}{\partial b_i^{(3)}} = \frac{\partial \mathcal{L}}{\partial x_i^{(3)}} \frac{\partial x_i^{(3)}}{\partial z_i^{(3)}} \frac{\partial z_i^{(3)}}{\partial b_i^{(3)}} = -(y_i - x_i^{(3)}) \mathrm{f}'^{(3)}\left(\mathrm{z}_\mathrm{i}^{(3)}\right)$$

Gradient for Layer 2

- Stochastic gradient for $W_{jk}^{(2)}$



  - *Recall:*
    - $\mathcal{L} = \frac{1}{2}\left\|\boldsymbol{y} - \boldsymbol{x}^{(3)}\right\|^2 = \frac{1}{2}\sum_i \left(y_i - x_i^{(3)}\right)^2$
    - $x_i^{(3)} = f^{(3)}\left(z_i^{(3)}\right);\ z_i^{(3)} = \sum_j W_{ij}^{(3)} x_j^{(2)} + b_i^{(3)}$
    - $x_j^{(2)} = f^{(2)}\left(z_j^{(2)}\right);\ z_j^{(2)} = \sum_k W_{jk}^{(2)} x_k^{(1)} + b_j^{(2)}$

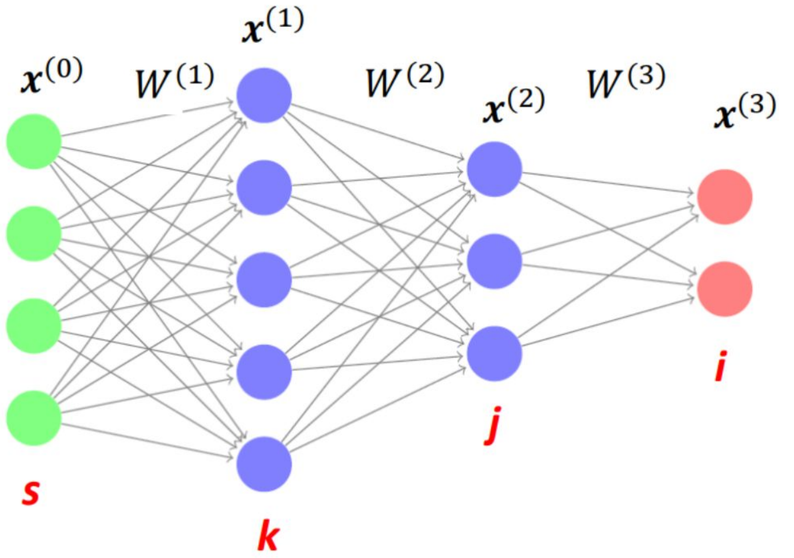
- $$\frac{\partial \mathcal{L}}{\partial W_{jk}^{(2)}} = \sum_i \frac{\partial \mathcal{L}}{\partial x_i^{(3)}} \frac{\partial x_i^{(3)}}{\partial z_i^{(3)}} \frac{\partial z_i^{(3)}}{\partial x_j^{(2)}} \frac{\partial x_j^{(2)}}{\partial z_j^{(2)}} \frac{\partial z_j^{(2)}}{\partial W_{jk}^{(2)}}$$

$$= \sum_i \underbrace{\underbrace{-(y_i - x_i^{(3)}) \mathrm{f}'^{(3)}\left(\mathrm{z}_\mathrm{i}^{(3)}\right)}_{\delta_i^{(3)}} \mathrm{w}_{ij}^{(3)} \mathrm{f}'^{(2)}\left(\mathrm{z}_\mathrm{j}^{(2)}\right) x_k^{(1)}}_{\delta_j^{(2)}}$$

Gradient for Layer 1

- Stochastic gradient for $W_{ks}^{(1)}$

$$\frac{\partial \mathcal{L}}{\partial W_{ks}^{(1)}} = \sum_j \delta_j^{(2)} W_{jk}^{(2)} f'^{(1)}\left(z_k^{(1)}\right) x_s^{(0)}$$



$$\delta^{(l)} = \left(W^{(l+1)T}\delta^{(l+1)}\right) \odot f^{(l)\prime}\left(z^{(l)}\right)$$

$$\frac{\partial L}{\partial W^{(l)}} = \delta^{(l)}\left(x^{(l-1)}\right)^T$$

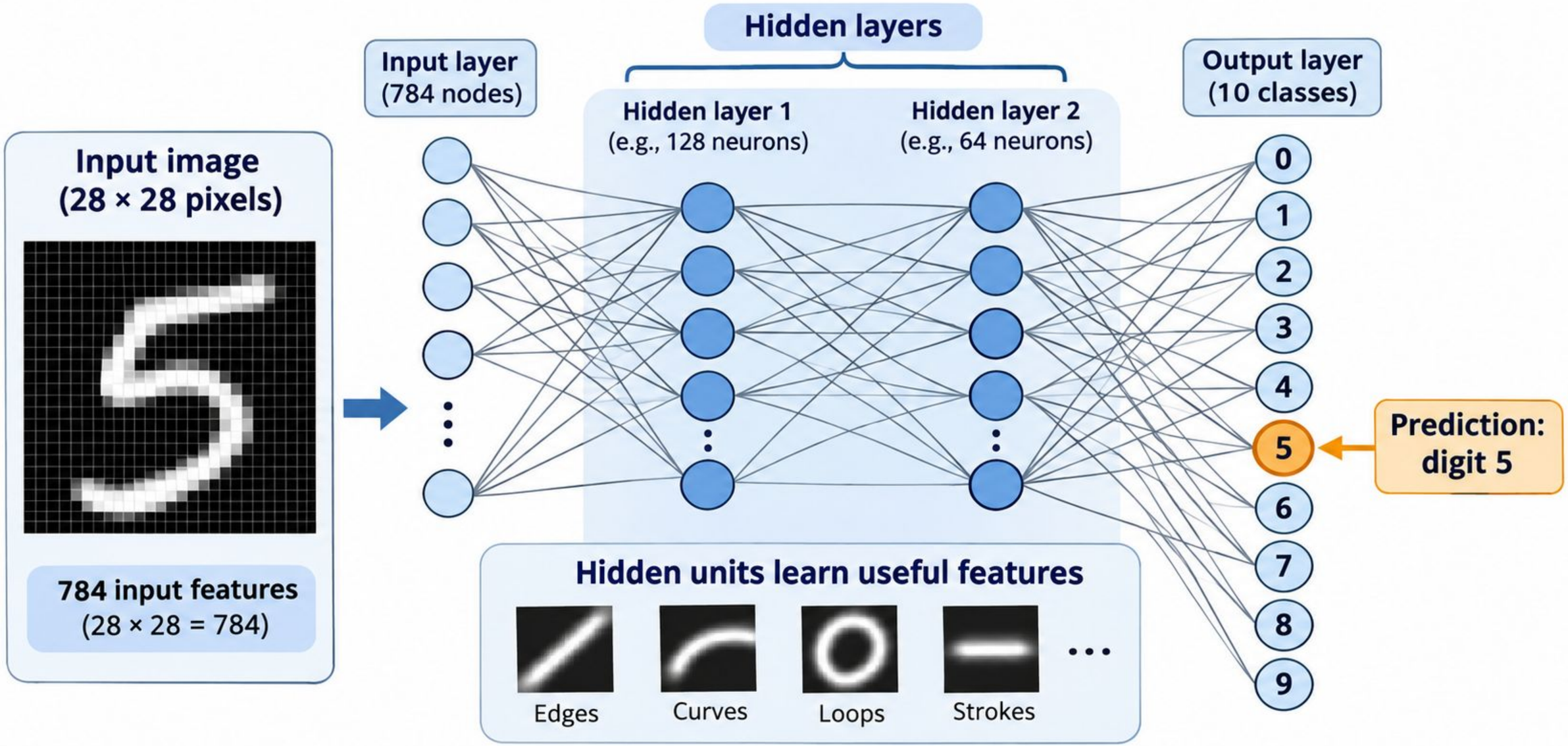
Backpropagation Steps to Learn Weights

- Initialize weights to small random numbers, associated with biases
- **Repeat** until a stopping criterion is met.
- **For** each training example
  - **Propagate the inputs forward** (by applying activation function)
    - For layer l = 1: L
      - Calculate $z^{(l)} = W^{(l)} xx^{(l-1)} + b^{(l)}$
      - Calculate $x^{(l)} = f^{(l)} z^{(l)}$ (elementwise activation)
  - **Backpropagate the error** (by updating weights and biases using Gradient Descent / SGD)
    - Calculate $\delta^{(L)}$, Update $W^{(L)}$ and $b^{(L)}$ based on $\frac{\partial \mathcal{L}}{\partial W^{(L)}} = \delta^{(L)} (x^{(L-1)})^T$ and $\frac{\partial \mathcal{L}}{\partial b^{(L)}} = \delta^{(L)}$
    - For layer l = L-1: 1
      - Calculate $\delta^{(l)} = \left(W^{(l+1)}\right)^T \delta^{(l+1)} \circ f'^{(l)}\left(z^{(l)}\right)$
      - Update $W^{(L)}$ and $b^{(L)}$ based on $\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} (x^{(l-1)})^T$ and $\frac{\partial \mathcal{L}}{\partial b^{(l)}} = \delta^{(l)}$
- Terminating condition (convergence, max iteration, etc.)

Strengths and Limitations of Neural Networks

- Limitations
  - Can require significant training time and computation
  - Many hyperparameters to choose
    - network depth
    - number of hidden units
    - learning rate
    - regularization
  - Often difficult to interpret learned weights and hidden representations
- Strengths
  - Can learn complex nonlinear relationships
  - Successful in many real-world applications: image recognition; speech; language
  - Can learn useful representations automatically
  - Computation can be highly parallelized
  - Deep networks can model very complex functions

Example: Handwritten Digits Recognition



Summary

- A neural network is built from connected computational units.
- Each unit computes a weighted sum and applies an activation function:

$$z = w^T x + b, \quad \alpha = f(z)$$

- Nonlinear activation functions allow neural networks to model complex relationships.
- The loss function measures prediction error.
- Backpropagation uses the chain rule to compute gradients.
- Gradient Descent / SGD uses those gradients to update the parameters.
- Regularization helps reduce overfitting and improve generalization.
- Hidden layers can learn useful intermediate representations from data.